

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ



UPA - 2. projekt

Ukládání rozsáhlých dat v NoSQL databázích

Tým xvolfr00

Obsah

1	Analýza NoSQL databází	2
1.1	Sloupcová wide-column databáze	2
1.2	Dokumentová databáze	2
1.3	Grafová databáze	3
1.4	Databáze časových řad	3
2	Využití konkrétních datových sad pro každý typ databáze	4
2.1	Sloupcová wide-column databáze	4
2.1.1	Struktura datové sady	4
2.1.2	Definice úložiště	5
2.1.3	Import dat	5
2.1.4	Dotaz nad databází	6
2.2	Dokumentová databáze	6
2.2.1	Struktura datové sady	6
2.2.2	Proč je použití MongoDB vhodné	7
2.2.3	Definice úložiště	7
2.2.4	Import dat	8
2.2.5	Dotaz	8
2.2.6	Zpracování dotazu	9
2.3	Grafová databáze	9
2.3.1	Struktura datové sady	9
2.3.2	Proč je použití Neo4J vhodné	10
2.3.3	Definice úložiště	10
2.3.4	Import dat	10
2.3.5	Dotaz	11
2.4	Databáze časových řad	11
2.4.1	Struktura datové sady	12
2.4.2	Proč je použití InfluxDB vhodné	12
2.4.3	Definice úložiště	12
2.4.4	Import dat	12
2.4.5	Dotaz	13

1 Analýza NoSQL databází

NoSQL databáze jsou nerelační databázové systémy navržené pro práci s velkými objemy dat. Ta bývají většinou v nestrukturované podobě. Hodí se pro projekty, kde je důležitá flexibilita a škálovatelnost schématu a rychlost při práci s velkým objemem dat. Podle určení se dělí na následující kategorie.

1.1 Sloupcová wide-column databáze

Struktura dat

Wide-column databáze, jako například Apache Cassandra, je druh NoSQL databáze, která organizuje ukládání dat do flexibilních sloupců, které lze rozložit na více serverů nebo databázových uzlů. Na rozdíl od tradičních relačních databází jsou navrženy pro zpracování velkého objemu nestrukturovaných dat.

Data jsou strukturována do Keyspace, který seskupuje související tabulky, Primárního klíče a Partition, která zajišťuje distribuci dat mezi uzly.

Příklady použití

Wide-column databáze jsou ideální pro případy použití, které vyžadují velké množství dat, které lze distribuovat mezi více databázových uzlů, zejména pokud sloupce nejsou pro každý řádek vždy stejné. Příklady zahrnují údaje z logů, data ze senzorů IoT, časové řady nebo data založená na attributech.

Výhody

Mezi výhody wide-column NoSQL databází patří rychlost dotazování, škálovatelnost a flexibilní datový model.¹

Nevýhody

Mezi nevýhody wide-column NoSQL databází patří omezená podpora pokročilých funkcí, problematická migrace dat a omezené možnosti datového modelování.²

1.2 Dokumentová databáze

Struktura dat

Tento druh databází ukládá data do tzv. dokumentů, což je struktura, kde jsou jednotlivé položky uloženy jako páry klíč-hodnota. Dokumenty popisující tentýž druh dat se ukládají do kolekcí. Dokumenty jsou uloženy v serializačním formátu, např. *JSON*, *XML* nebo *YAML*.

Příklady použití

Ukládání a zpětné získávání velkého množství nestrukturovaných či proměnlivých dat, např. CMS (content management system), ukládání uživatelem vytvořeného obsahu na sociálních sítích.

Výhody

- Vysoká úroveň flexibility, dokumenty mohou mít v rámci kolekce proměnlivou strukturu, tudíž nenastanou problémy při změně schématu dokumentu.
- Škálovatelnost, dokumentové databáze jsou schopny agresivního horizontálního škálování, což zlepšuje dostupnost. I při vyřazení několika uzlů v clusteru je databáze stále funkční.

¹<https://www.scylladb.com/glossary/wide-column-database/>

²<https://databasetown.com/wide-column-database-use-cases/>

Nevýhody

- Slabá podpora pro kontrolu referenční integrity, kvůli umožněné flexibilitě není nad ukládanými daty prováděno velké množství kontrol, které znemožní uložit nesprávná, či nějakým způsobem poškozená data.
- Vyšší úroveň redundance. Vlivem ukládání dat jako jednotek, jež obsahují všechna data potřebná popisu sebe sama, vzniká problém s replikací dat na více místech.
- Slabá podpora složitých dotazů. Díky existující, byť slabé, podpoře pro vzájemné reference je možné provádět dotazy přes vícero kolekcí, avšak to zpravidla není vhodné kvůli slabé podpoře integrity dat, např. dokument s daným ID nemusí existovat.

1.3 Grafová databáze

Struktura dat

Grafová databáze, jako je Neo4j, je druh NoSQL databáze, která organizuje data ve formě uzlů (nodes), hran (edges) a vlastností (properties). Uzly reprezentují entity (např. osoby nebo produkty), hrany vyjadřují vztahy mezi těmito entitami, a vlastnosti ukládají konkrétní atributy uzlů a hran. Tato struktura umožňuje přímé propojení mezi uzly, což zrychluje a usnadňuje vyhledávání složitých vztahů v datech.

Příklady použití

Grafové databáze se používají v aplikacích, kde je důležité modelovat složité sítě a vztahy mezi entitami. Typické příklady zahrnují sociální sítě, doporučovací systémy, a správu sítí (reprezentace a monitorování počítačových sítí).

Výhody

- Rychlé vyhledávání v datech s komplexními vztahy – grafové databáze umožňují efektivní navigaci mezi uzly a hranami díky přímým propojením.
- Flexibilní modelování vztahů – grafy mohou jednoduše modelovat složité vztahy a hierarchie, které by v relačních databázích byly těžko realizovatelné.
- Výborný výkon u dotazů typu "procházení grafu" – díky optimalizované struktuře grafové databáze efektivně pracují s dotazy, které vyžadují sledování cest mezi uzly, například při hledání spojení mezi uživateli nebo objektů ve velkých sítích.

Nevýhody

- Omezená podpora složitých transakcí – grafové databáze obecně neposkytují stejné možnosti zpracování transakcí jako relační databáze, což může být limitující pro některé aplikace.
- Obtížnější škálování – škálování grafových databází je obvykle náročnější než u jiných typů NoSQL databází, jelikož je třeba udržet konzistenci vztahů mezi uzly napříč uzly clusteru.
- Menší podpora v porovnání s relačními databázemi – vzhledem k tomu, že jsou grafové databáze méně rozšířené, může být obtížnější najít odborníky nebo nástroje pro správu a vývoj.

1.4 Databáze časových řad

Tyto databáze jsou určené pro práci s velkým množstvím časově orientovaných dat.

Struktura dat

Data jsou ukládána podle času nestrukturovaně a spojování tabulek není podporováno. To zajišťuje velkou efektivitu při vyhledávání dat podle času. Klíč záznamu se skládá z časového údaje a sloupců označených jako tagy. Nad tagy jsou vytvářeny indexy, které zajišťují rychlé vyhledávání.

Příklady použití

Monitorování systémů (např. sledování výkonnosti serverů), IoT, analýza finančních dat, měření v reálném čase (např. senzory).

Výhody

Optimalizováno pro efektivní zápis a čtení časových dat. Nabízí funkce jako automatickou agregaci, downsampling a vyhledávání trendů v datech.

Nevýhody

Méně vhodné pro složité vztahy mezi daty nebo strukturovaná data, která nejsou časově závislá. Omezenější možnosti při dotazování na obecná data, která nejsou založena na čase.

Příklady databází

InfluxDB: Jeden z nejznámějších systémů pro ukládání a analýzu časových řad, široce využívaný pro monitorování systémů a IoT.

Prometheus: Open-source nástroj zaměřený na monitorování a alerting, často používaný v DevOps prostředí.

2 Využití konkrétních datových sad pro každý typ databáze

Datové sady jsou k dispozici na odkazech Datová sada Ostrava a Statutární město Ostrava (212).

2.1 Sloupcová wide-column databáze

Pro demonstraci práce se sloupcovou wide-column databází byla zvolena Apache Cassandra. Jako demonstrační datovou sadu jsme si vybrali Kriminalita a přestupky³. Tato datová sada je uložena ve formátu CSV.

2.1.1 Struktura datové sady

- **Rok** – Kalendářní rok
- **Měsíc** – Kalendářní měsíc
- **Obvod** – Městský obvod
- **Kód obvodu** – Kód městského obvodu podle RUIAN
- **Události** – Počet událostí. Zahrnuje veřejný pořádek, podezřelé osoby, občan v nouzi, ...
- **Dopravní přestupky** – Počet dopravních přestupků
- **Ostatní přestupky** – Počet ostatních přestupků

³<https://opendata.ostrava.cz/category/kriminalita-prestupky/>

- **Preventivní akce** – Počet preventivních akcí

Cassandra je běžně používaná v aplikacích, kde se provádí několik milionů dotazů za sekundu, což je možné díky její vysoké škálovatelnosti. V našem případě ji demonstrujeme na menší datové sadě. Na vybrané datové sadě můžeme efektivně a rychle zpracovávat zápisy a čtení většího množství dat. Cassandra také umožňuje načítat pouze ty sloupce, se kterými chceme pracovat, díky čemuž dosahujeme efektivního výpočtu. V případě potřeby umožňuje flexibilní úpravy dat a struktury databáze, což se při vybrané sadě hodí, protože tato datová sada je aktualizována každý měsíc. Příkazy v následujících sekcích budou vytvořeny v CQL Shellu pro Apache Cassandra.

2.1.2 Definice úložiště

V Apache Cassandra musíme nejprve vytvořit keyspace. V rámci keyspace můžeme definovat replikační faktor, jehož hodnota určuje počet kopií údajů mezi uzly. My budeme pracovat s jedním uzlem, takže hodnotu replikačního faktoru nastavíme na 1. Pomocí příkazu `USE` budeme dále s vytvořením keyspace provádět další operace. Na následující ukázce můžeme vidět příklad vytvoření keyspace:

```
CREATE KEYSPACE IF NOT EXISTS kriminalita WITH
    REPLICATION = { 'class': 'SimpleStrategy', 'replication_factor': 1 };
USE kriminalita;
```

Dále, podobně jako v relačních databázích, musíme nadefinovat tabulky. Určíme si primární klíč a datové typy pro jednotlivé sloupce. Primární klíč může být rozdělen na partition klíč a clustering klíč. Partition klíč rozděluje data mezi uzly, zatímco clustering klíče určují pořadí řádků v rámci každé partition. V našem případě jsme si za partition klíč zvolili (rok, kod_obvodu) a za clustering klíč měsíc, kterým budou seřazeny řádky obsahující stejný rok a kód obvodu. Po prozkoumání datové sady jsme zjistili, že tento klíč je postačující a hlavně unikátní pro rozlišení jednotlivých záznamů. Na následující ukázce můžeme vidět příklad vytvoření tabulky:

```
CREATE TABLE IF NOT EXISTS prestupky (
    rok int,
    mesic int,
    obvod text,
    kod_obvodu text,
    udalost int,
    dopravni_prestupky int,
    ostatni int,
    preventivni_akce int,
    PRIMARY KEY ((rok, kod_obvodu), mesic)
);
```

2.1.3 Import dat

Po vytvoření tabulek musíme do databáze nějakým způsobem vložit data. Nejprve si CSV soubor datové sady stáhneme. Před samotným vložením upravíme data pomocí python skriptu:

```
new_header = "rok,mesic,obvod,kod_obvodu,udalost,
    dopravni_prestupky,ostatni,preventivni_akce\n"

with open('20220101_20240901_mp.csv', 'r', encoding='utf-8') as file:
    lines = file.readlines()
```

```

lines[0] = new_header

for i in range(1, len(lines)):
    lines[i] = lines[i].replace('\N', '')

with open('20220101_20240901_mp.csv', 'w', encoding='utf-8') as file:
    file.writelines(lines)

```

Nejprve změníme hlavičku datové sady, aby neobsahovala diakritiku. V datové sadě v některých sloupcích chybí hodnoty. Namísto číselné hodnoty se v těchto místech nachází hodnota \N, kterou nahradíme prázdnou hodnotou. Potom pomocí příkazu COPY data vložíme do databáze, jak můžeme vidět na následující ukázce:

```

COPY prestupky (rok, mesic, obvod, kod_obvodu, udalost,
dopravni_prestupky, ostatni, preventivni_akce)
FROM '20220101_20240901_mp.csv' WITH HEADER = true;

```

2.1.4 Dotaz nad databází

Dotazujeme se podobně jako v relačních databázích za použití SELECT. Následující ukázka obsahuje příklad dotazu, který sečte počet dopravních přestupků a preventivních akcí pro rok 2023 a kód obvodu 546135, což reprezentuje obvod Ostrava-Jih:

```

SELECT rok,
obvod,
SUM(dopravni_prestupky) AS "dopravni prestupky",
SUM(preventivni_akce) AS "preventivni akce"
FROM prestupky WHERE kod_obvodu='546135' AND rok=2023;

```

Tento dotaz nevyžaduje další filtrování přes argument ALLOW FILTERING, protože vyhledávání probíhá za použití partition klíče. Kdyby bylo nutné filtrování, zpomalilo by to výkon databáze a ztratily by se výhody Apache Cassandra, které souvisí s její schopností efektivně zpracovávat velké objemy dat. Pokud by těchto dotazů bylo mnoho, doporučuje se přeuspořádat strukturu tabulky pro optimalizaci výkonu.

2.2 Dokumentová databáze

Pro data k uložení do dokumentové databáze byla vybrána sada Dopravní přestupky dle data a místa spáchání v roce 2024, která je uložena ve formátu CSV. Jako konkrétní implementace dokumentové databáze byla vybrána databáze MongoDB.

2.2.1 Struktura datové sady

- **Číslo případu** – Číslo daného přestupku. Pokud má přestupek více právních kvalifikací, je rozepsán do více řádků pod stejným číslem případu.
- **Datum spáchání** – Datum spáchání přestupku ve formátu RRRR-MM-DD
- **Místo činu** – Upřesnění místa činu
- **Číslo zákona** – Číslo zákona
- **Paragraf** – Paragraf příslušného zákona
- **Odstavec** – Odstavec příslušného paragrafu zákona

- **Písmeno** – Písmeno příslušného odstavce paragrafu zákona
- **Bod** – Bod příslušného písmena odstavce paragrafu zákona
- **Způsob vyřízení** – Způsob vyřízení (stav vyřízení k poslednímu dni daného období, „neuvedeno“ - přestupek je dosud v řešení)

2.2.2 Proč je použití MongoDB vhodné

Tato datová sada je vhodná pro uložení do dokumentové databáze MongoDB, protože:

1. Jednotlivé záznamy neobsahují reference na jiné záznamy, což je dobré, protože udržování komplexních vztahů není silnou stránkou dokumentové databáze (je vhodnější uložit data plochým způsobem).
2. Evidence přestupků má poměrně dobrou šanci být rozšířena mimo dané působení (například rozšířena na celou republiku), což by nebyl pro dokumentovou databázi problém, díky jednoduché škálovatelnosti.
3. Při změně struktury odkazovaného zákona není třeba velkých změn, nové záznamy jednoduše použijí nové schéma.
4. MongoDB podporuje jednoduchou validaci, dostatečnou pro tento typ dat, aniž by bylo nutné vzdát se jednoduchosti a z ní pramenící flexibility databáze.

2.2.3 Definice úložiště

MongoDB ze své podstaty definic schémat nepodporuje. Je však možné definovat validační pravidla, která zabrání vložení dat, která by byla ve špatném formátu. Následující příkazy v Mongo Shellu vytvoří kolekci `trafficViolations` s jednoduchou validací, která pouze vyžaduje určité atributy pro každý dokument. Vynucené atributy vychází z povahy datové sady.

Po vytvoření kolekce se nad kolekcí vytvoří index podle Čísla případu, aby bylo dotazování, či následné operace update rychlejší. Nakonec je možné uložit záznam do databáze přímo z Mongo Shellu.

```
db.createCollection("trafficViolations", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      title: "trafficViolations schema validation",
      required: [
        "Číslo případu",
        "Datum spáchání",
        "Číslo zákona",
        "Paragraf",
        "Odstavec",
        "Způsob vyřízení"
      ]
    }
  }
})
db.trafficViolations.createIndex({ "Číslo případu":1 })
db.trafficViolations.insertOne({
  "Číslo případu": 14,
  "Datum spáchání": "2023-10-30",
```

```

    "Místo činu": "Nejpěknější místo v Ostravě",
    "Číslo zákona": "361/2000",
    "Paragraf": "125c",
    "Odstavec": 1,
    "Písmeno": "k)",
    "Způsob vyřízení": "příkazový blok",
  })
})

```

2.2.4 Import dat

Databáze MongoDB poskytuje nástroj `mongoimport`, který je schopen importovat do databáze data ve formátu *JSON*, *CSV* nebo *TSV*.

V datové sadě se vyskytuje množství řádků se syntakticky chybným vnořením uvozovek, kvůli kterým nástroj `mongoimport` selže při parsování *CSV* dat. Tyto vnořené uvozovky lze odstranit například pomocí unixového příkazu `sed` za použití regulárních výrazů.

```

sed -i -E \
's/" ([^"]*)" ([^"]*)" ([^"]*)" /"\1 \2 \3"/g; \
s/" ([^"]*)" ([^"]*)" /"\1 \2"/g' \
/path/to/data.csv;

```

Nástrojem `mongoimport` je posléze možné importovat data z *CSV* souboru do nově vytvořené databáze. Následující příkaz provede operaci „`upsert`“, s tím, že nahrazuje záznamy se shodným číslem případu. Tedy, je možné tento příkaz použít jak pro počáteční naplnění, tak pro pozdější nahrazování a modifikaci záznamů.

```

mongoimport --host=<HOST>\
--db=<DATABASE>\
--collection=trafficViolations\
--type=csv\
--headerline\
--mode=upsert\
--upsertFields='Číslo případu'\
/path/to/data.csv;

```

2.2.5 Dotaz

```

db.trafficViolations.aggregate([
  {
    $match: {
      $or: [
        {"Číslo zákona": "361/2000"},
        {"Číslo zákona": "168/1999"},
      ]
    }
  },
  {
    $group: {
      _id: "$Datum spáchání",
      "Počet spáchání": {$count: {}},
    }
  },
])

```

```

    {
      $sort: {
        "Počet spáchání": -1
      }
    }
  ] )

```

Tento dotaz realizuje distribuovaný dotaz pomocí agregační pipeline. Vráť seřazený seznam dvojic (datum; počet), seřazený podle počtu sestupně, kde každý počet značí kolik bylo v daný den evidováno přestupků porušujících zákon 361/2000 nebo 168/1999.

2.2.6 Zpracování dotazu

Uvažujme MongoDB databázi skládající se z clusteru tzv. shards, což jsou jednotlivé servery uchovávající data. V tomto clusteru se nachází centrální server, na kterém běží proces mongos, tento server se nazývá Router.

Jakmile mongos dostane dotaz, nejdříve zjistí, jakým shards má daný dotaz poslat. Kolekce dokumentů totiž může být distribuována přes více shards. Tuto informaci zjistí buď pomocí tzv. shard keys, což jsou klíče identifikující shard, na kterém je dokument uložen, nebo musí dotazovat všechny shards.

Při vykonávání této agregační pipeline lze vykonat první fázi (match) nezávisle na všech uzlech, aniž by spolu musely komunikovat. V dalších fázích (group a sort) již probíhá komunikace mezi jednotlivými uzly, které si vyměňují data při vykonávání dotazu.

Nakonec jsou tato data agregována na Routeru a odpověď je poslána klientovi, který s daty naloží podle svého uvážení.

2.3 Grafová databáze

Pro demonstraci práce se grafickou databází byla zvolena Neo4j. Jako demonstrační datovou sadu jsme si vybrali Pčitová mapa. Místa, která doporučí návštěvníkům odjinud.. Tato datová sada je uložena ve formátu CSV.

2.3.1 Struktura datové sady

- **ID** – Jedinečné identifikační číslo respondenta - stejné pro všechny odpovědi respondenta.
- **NAZOR** – Názor respondenta ke konkrétnímu zadanému bodu.
- **DATUM** – Datum vložení.
- **OBVOD** – Název městského obvodu, ve kterém je bod umístěn.
- **X_JTSK** – Souřadnice S-JTSK
- **Y_JTSK** – Souřadnice S-JTSK
- **X_WGS84** – Souřadnice WGS 84
- **Y_WGS84** – Souřadnice WGS 84
- **POHLAVI** – Pohlaví respondenta (M - muž, Z - žena)
- **ZIJU** – Městský obvod, ve kterém respondent bydlí.
- **VOLNY_CAS** – Městský obvod, ve kterém respondent tráví většinu volného času.
- **VEK** – Věková kategorie respondenta.

2.3.2 Proč je použití Neo4J vhodné

Neo4j je grafová databáze, která se hodí pro aplikace, kde je důležité modelovat složité vztahy mezi daty, jako jsou například sociální sítě, doporučovací systémy nebo geografické informační systémy. Grafové databáze umožňují intuitivní a efektivní reprezentaci dat ve formě uzlů (nodes) a vztahů (relationships), což umožňuje efektivní práci s propojenými daty.

Vybraná datová sada nám umožňuje modelovat a vizualizovat vztahy a závislosti mezi jednotlivými městskými obvody pomocí grafové databáze. Na základě parametrů, jako jsou bydliště, volný čas, věková kategorie a názory respondentů, můžeme identifikovat souvislosti mezi obvody. Každý názor respondenta je uložen jako samostatný uzel, což umožňuje sledovat různé pohledy a postoje obyvatel v jednotlivých obvodech. Tímto způsobem lze lépe porozumět sociálním a demografickým trendům v každém obvodu i mezi nimi, což je důvodem pro použití grafové databáze.

Dotazy se píšou v jazyce Cypher, který umožňuje specifikovat, které uzly a vztahy nás zajímají.

2.3.3 Definice úložiště

V Neo4j se data ukládají jako uzly a vztahy, které vytváříme přímo v grafu. Pro nastavení datové struktury definujeme uzly pro každou entitu a nastavujeme mezi nimi vztahy a k nim přiřazujeme informace o těchto entitách. Pro definici schématu lze v jazyce CQL definovat typy jednotlivých uzlu a vztahu pomocí klauzule CREATE. Použití CREATE můžeme vidět na následující ukázce:

```
CREATE (n:Nazor {
  id: apoc.create.uuid(),
  text: $nazor,
  datum: $datum,
  x_jtsk: $x_jtsk,
  y_jtsk: $y_jtsk,
  x_wgs84: $x_wgs84,
  y_wgs84: $y_wgs84
})
```

2.3.4 Import dat

Data je nutné nejprve transformovat a následně uložit jako graf v databázi. Bude obsahovat dva uzly – Obvod a Respondent. Ty se propojí hranami podle odpovědí respondentů a vztahu k obvodu, kde jsou uvedeny. Každý názor respondenta je uložen jako samostatný uzel (Nazor), propojený s konkrétním respondentem i s obvodem, kde byl názor zaznamenán.

```
import pandas as pd
from neo4j import GraphDatabase

# Načtení dat z CSV
data = pd.read_csv("raw_data.csv")

driver = GraphDatabase.driver(uri, auth=(username, password))

# Funkce pro vložení dat
def insert_data(tx, record):
    tx.run("""
        MERGE (o:Obvod {name: $obvod})
```

```

MERGE (r:Respondent {id: $id})
SET
    r.pohlavi = $pohlavi,
    r.ziju = $ziju,
    r.volny_cas = $volny_cas,
    r.vek = $vek

CREATE (n:Nazor {
    id: apoc.create.uuid(),
    text: $nazor,
    datum: $datum,
    x_jtsk: $x_jtsk,
    y_jtsk: $y_jtsk,
    x_wgs84: $x_wgs84,
    y_wgs84: $y_wgs84
})

MERGE (r)-[:HAS_OPINION]->(n)
MERGE (n)-[:LOCATED_IN]->(o)
""", id=record['ID'], nazor=record['NAZOR'], datum=record['DATUM'],
obvod=record['OBVOD'], x_jtsk=record['X_JTSK'],
y_jtsk=record['Y_JTSK'], x_wgs84=record['X_WGS84'],
y_wgs84=record['Y_WGS84'], pohlavi=record['POHLAVI'],
ziju=record['ZIJU'], volny_cas=record['VOLNY_CAS'],
vek=record['VEK'])

# Vložení všech dat
with driver.session() as session:
    for _, row in data.iterrows():
        session.write_transaction(insert_data, row)

# Uzavření připojení
driver.close()

```

2.3.5 Dotaz

Dotazy v Neo4j se provádějí pomocí klauzule MATCH, která se používá pro definici vzoru v grafu, obdobně jako příkaz SELECT v SQL. Následující dotaz demonstruje získání názorů o obvodu *Moravská Ostrava a Přívoz* u lidí ve věkovém rozmezí 25,–,35 let.

```

MATCH (r:Respondent)-[:HAS_OPINION]->(n:Nazor)-[:LOCATED_IN]->(
    o:Obvod {name: "Moravská Ostrava a Přívoz"})
WHERE r.vek = "25 - 35"
RETURN n.text AS Názor, r.id AS RespondentID, r.vek AS Věk,
    o.name AS Obvod

```

2.4 Databáze časových řad

Pro data k uložení do databáze časových řad byla vybrána sada *Statistika průjezdu vozidel ze sledovaných křižovatek*, která je uložena ve formátu CSV.

2.4.1 Struktura datové sady

- **Datum:** Datum a čas pořízení záznamu ve formátu *DD.MM.YYYY HH:mm*.
- **Stanice:** Název křižovatky, na které byl záznam pořízen.
- **Třída objektu:** Klasifikace podle velikost vozidla.
- **Počet:** Počet vozidel dané třídy, která v danou hodinu projela křižovatkou.

2.4.2 Proč je použití InfluxDB vhodné

InfluxDB je ideální pro ukládání dat o průjezdu vozidel na křižovatkách díky své specializaci na časové řady. To umožňuje efektivní ukládání a zpracování dat s časovými razítky. Databáze podporuje rychlé dotazy, agregace a filtrace podle času. Díky optimalizaci pro rychlé zápisy a nízké latenci čtení je vhodná i pro analýzu v reálném čase. InfluxDB usnadňuje dlouhodobé sledování a archivaci velkých objemů dat z různých křižovatek.

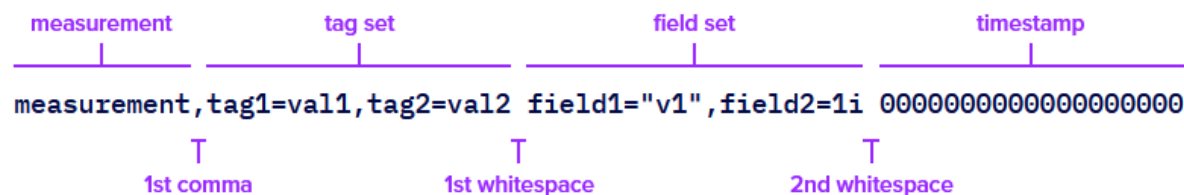
2.4.3 Definice úložiště

V InfluxDB jsou data ukládána jako měření (measurements) v podobě časových řad (time series), kde každá časová řada představuje jednu konkrétní metriku nebo typ dat. Struktura dat obsahuje následující klíčové prvky:

- **Měření (Measurement):** Představuje název měřených dat (identifikátor).
- **Časové razítko (Timestamp):** Každý záznam v InfluxDB obsahuje přesné časové razítko, které udává, kdy byla data naměřena nebo zaznamenána.
- **Tagy (Tags):** Jsou to páry klíč-hodnota, které slouží k efektivní identifikaci nebo filtrování dat. Typicky mohou zahrnovat statické informace, jako je název křižovatky, druh vozidla, nebo směrování dopravního toku. Tagy jsou indexované, což usnadňuje rychlé dotazy a agregace.
- **Pole (Fields):** Představují naměřené hodnoty, které mohou být číselné, textové nebo logické. Na rozdíl od tagů nejsou pole indexovaná, ale jsou určena pro uložení datových hodnot, jako je počet vozidel nebo rychlost.
- **Série (Series):** Je kombinací měření a konkrétních hodnot tagů. Každá unikátní kombinace měření a hodnot tagů tvoří vlastní časovou řadu, která obsahuje sekvenci hodnot pole v čase.

2.4.4 Import dat

Pro přidání záznamu měření se používá *Line Protocol*. Jeden řádek odpovídá jednomu měření a skládá se ze 4 částí – identifikátor měření (measurement), tagy, pole a časové razítko.



Uložení jednoho měření na křižovatce by vypadalo takto:

```
INSERT traffic,stanice="nazev krizovatky",trida_objektu="Ostatní"
pocet=321 1704294000
```

Časová razítka v poskytnutém CSV souboru jsou ale ve formátu *DD.MM.YYYY HH:mm*, který *Line Protocol* nepodporuje. Je tedy nejprve nutné změnit jejich formát na timestamp například pomocí následujícího *Python* skriptu. Také upravíme názvy sloupců, aby neobsahovaly mezery a diakritiku.

```
import pandas as pd
from datetime import datetime

data = pd.read_csv("raw_data.csv")

data.rename(columns={
    'Datum': 'date',
    'Stanice': 'station',
    'Třída objektu': 'cls',
    'Počet': 'cnt'
}, inplace=True)

data['date'] = data['date'].apply(
    lambda x: int(datetime.strptime(x, "%d.%m.%Y %H:%M").timestamp())
)

data.to_csv("data.csv", index=False)
```

Aby se nemusel přidávat každý záznam zvlášť, existuje příkaz pro import dat ze souboru.

```
influx write \
--bucket "intersections-traffic" \
--format=csv \
--file data.csv \
--header "#constant measurement,intersection" \
--header "#datatype dateTime:,tag,tag,int"
```

2.4.5 Dotaz

Struktura dotazů je založena na jazyce *SQL*, který je rozšířen o práci s časovými údaji. Pro demonstraci lze použít dotaz, který vrátí střední hodnotu počtu průjezdů aut pro každou křižovatku během předchozího dne. Data jsou seskupena podle názvu křižovatky (*station*) s přesností na jednu hodinu.

```
SELECT mean("cnt")
FROM "traffic"
WHERE time >= now() - 24h
GROUP BY "station", time(1h)
```